

GlobalRetail ETL — Documentación Completa del Proyecto

Asignatura: Data Engineering (DAN2 — 2C)
Proyecto: Capstone — Modern Data Stack ETL Pipeline
Equipo: Group 5 — SEQUERA GÓMEZ, Manuel · GONZALEZ GRANDE, Angel · AGUILAR DE BELVA, Manuel María · DE ÁVILA RETES, Jose Antonio · CARRIÓN IZQUIERDO, Carlos Miguel
Stack: Apache Airflow 2.9 · Docker · Python 3.11 · pandas · PyMongo · MongoDB Atlas · Power BI
Fecha de entrega: Abril 2026

Índice

- 1. Contexto de negocio
- 2. Objetivos técnicos
- 3. Arquitectura del sistema
- 4. Configuración del entorno
- 5. Pipeline ETL — Detalle por fases
 - 5.1 Extract
 - 5.2 Transform
 - 5.3 Load
- 6. Extracción incremental — Watermarking
- 7. Idempotencia — bulk_write vs insert_many
- 8. Monitorización y manejo de errores
- 9. Calidad de datos
- 10. Dashboard Power BI
- 11. Decisiones de diseño y trade-offs
- 12. Guía de reproducibilidad
- 13. Conclusión

1. Contexto de negocio

GlobalRetail Corp es una empresa de e-commerce internacional que necesita un Business Intelligence dashboard para monitorizar su rendimiento global. Los datos están fragmentados en tres fuentes heterogéneas:

Fuente	Descripción	Formato
Sales Data	Transacciones de una tienda online (2010-2011)	CSV (Kaggle — Online Retail Dataset)
Geopolitical Data	Región y población por país	REST Countries API
Financial Data	Tipo de cambio GBP → EUR	Frankfurter API

El objetivo es consolidar estas tres fuentes en un único Data Warehouse centralizado (MongoDB Atlas) y exponer los datos a través de un dashboard de Power BI.

2. Objetivos técnicos

Objetivo	Solución implementada
Orquestación con Apache Airflow	DAG con TaskFlow API, schedule diario 02:00 UTC
Entorno containerizado	Docker Compose con Airflow Webserver, Scheduler y Postgres
Cloud Integration	MongoDB Atlas M0 Free Tier como Data Warehouse
Extracción incremental	Watermarking mediante colección <code>_pipeline_metadata</code>
Data Enrichment	Integración en tiempo real de REST Countries y Frankfurter
Observabilidad	Logging estructurado JSON + decorador <code>@timed</code>
Idempotencia	<code>bulk_write</code> con <code>UpdateOne(upsert=True)</code> + índice único compuesto

3. Arquitectura del sistema

Flujo de alto nivel

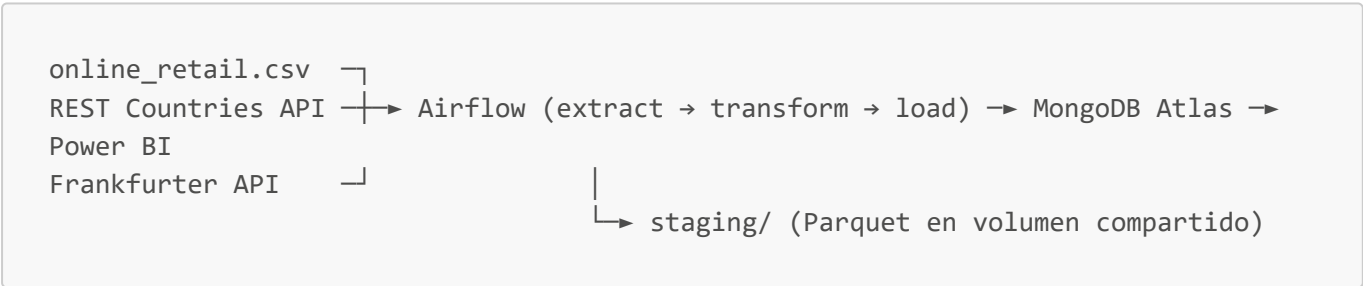


Diagrama de secuencia — una ejecución del DAG



```
ix_region)
    ↳ load_to_mongo
      | Crea índices (uq_business_key, ix_invoice_date,
      | bulk_write en batches de 1000 (upsert=True)
      ↳ Actualiza last_watermark = max(invoice_date)
```

Esquema del documento `fact_sales` en MongoDB

```
{
  "_id": "ObjectId(...)",
  "invoice_no": "536365",
  "stock_code": "85123a",
  "description": "white hanging heart t-light holder",
  "quantity": 6,
  "unit_price_gbp": 2.55,
  "invoice_date": "2010-12-01T08:26:00Z",
  "customer_id": "17850",
  "country": "united kingdom",
  "region": "Europe",
  "population": 67886011,
  "revenue_gbp": 15.30,
  "revenue_eur": 17.90,
  "fx_rate_gbp_eur": 1.170
}
```

Índices en la colección `fact_sales`

Nombre	Campos	Propósito
<code>uq_business_key</code> (único)	<code>invoice_no</code> , <code>stock_code</code> , <code>customer_id</code>	Idempotencia en upserts
<code>ix_invoice_date</code>	<code>invoice_date</code>	Consultas de watermark y filtros temporales en Power BI
<code>ix_region</code>	<code>region</code>	Agregación para gráfico de barras

4. Configuración del entorno

Requisitos previos

- Docker Desktop instalado y en ejecución
- Cuenta en MongoDB Atlas (cluster M0 Free Tier)
- Python 3.11+ con `pymongo`, `pandas`, `dnspython` (para Power BI)
- Power BI Desktop

Estructura del proyecto

```

globalretail-etl/
├── dags/
│   ├── globalretail_etl_dag.py    # DAG principal de Airflow
│   └── etl/
│       ├── config.py             # Configuración centralizada
│       ├── extract.py            # Fase Extract
│       ├── transform.py          # Fase Transform
│       ├── load.py               # Fase Load
│       └── logger.py             # Logging estructurado JSON
├── docs/
│   ├── architecture.md
│   ├── powerbi_setup.md
│   ├── technical_report.md
│   └── project_documentation.md  # Este documento
├── include/
│   ├── data/
│   │   └── online_retail.csv     # Dataset de Kaggle
│   └── staging/                 # Archivos Parquet intermedios
├── tests/
│   └── test_transform.py         # 7 tests unitarios
├── docker-compose.yml
├── Dockerfile
├── requirements.txt
└── .env                         # Variables de entorno (no en git)

```

Variables de entorno (.env)

```

AIRFLOW_ADMIN_USER=admin
AIRFLOW_ADMIN_PASSWORD=admin
AIRFLOW_UID=50000
MONGODB_URI=mongodb+srv://<usuario>:<password>@cluster0.xxxxx.mongodb.net/?
retryWrites=true&w=majority&appName=Cluster0
MONGODB_DATABASE=globalretail
MONGODB_COLLECTION=fact_sales

```

Servicios Docker Compose

Servicio	Imagen	Función
postgres	postgres:15-alpine	Backend de metadatos de Airflow
airflow-init	globalretail-airflow:2.9.3	Inicialización de BD y usuario admin (one-shot)
airflow-webserver	globalretail-airflow:2.9.3	UI web en http://localhost:8080
airflow-scheduler	globalretail-airflow:2.9.3	Programador y ejecutor de tareas

Comandos de arranque

```
# 1. Copiar y configurar variables de entorno
cp .env.example .env
# Editar .env con tu MONGODB_URI

# 2. Construir la imagen Docker
docker compose build

# 3. Inicializar la base de datos de Airflow (solo la primera vez)
docker compose up airflow-init

# 4. Levantar todos los servicios
docker compose up -d

# 5. Acceder a la UI de Airflow
# http://localhost:8080 → usuario: admin / contraseña: admin
```

5. Pipeline ETL — Detalle por fases

5.1 Extract

Archivo: `dags/etl/extract.py`

La fase de extracción realiza tres operaciones en paralelo:

a) Lectura del watermark (MongoDB)

```
def get_last_watermark(cfg: Config) -> datetime | None:
    with MongoClient(cfg.mongodb_uri) as client:
        meta = client[cfg.mongodb_database][cfg.metadata_collection]
        doc = meta.find_one({"_id": "last_watermark"})
        return doc["invoice_date"] if doc else None
```

En el primer run el watermark es `None` y se carga todo el CSV. En runs sucesivos solo se procesan las filas con `InvoiceDate > watermark`.

b) Lectura del CSV (incremental)

```
df = pd.read_csv(
    cfg.csv_path,
    encoding="ISO-8859-1",      # el dataset de Kaggle no es UTF-8
    dtype={"CustomerID": "string", "InvoiceNo": "string", ...},
    parse_dates=["InvoiceDate"],
)
if watermark is not None:
    df = df[df["InvoiceDate"] > watermark].copy()
```

Problema resuelto: El CSV usa encoding ISO-8859-1 (no UTF-8) y `CustomerID` tiene NaNs que rompen el cast a int. Solución: `encoding` explícito + `dtype` map.

c) APIs externas (con retry)

Ambas APIs usan `tenacity` con backoff exponencial:

```
@retry(stop=stop_after_attempt(5), wait=wait_exponential(multiplier=2, min=2,
max=16),
        retry=retry_if_exception_type(requests.RequestException), reraise=True)
def fetch_countries(cfg: Config) -> pd.DataFrame:
    resp = requests.get(cfg.countries_api_url, timeout=cfg.api_timeout_seconds)
    resp.raise_for_status()
    # extrae name, region, population de cada país
```

API	URL	Datos obtenidos
REST Countries	<code>https://restcountries.com/v3.1/all?fields=name,region,population</code>	region, population por país
Frankfurter	<code>https://api.frankfurter.dev/v1/latest?base=GBP&symbols=EUR</code>	tasa de cambio GBP→EUR

Los datos pesados (CSV filtrado + países) se escriben como **Parquet** en `include/staging/` y solo un manifiesto ligero (rutas + conteos) pasa por XCom a la siguiente tarea.

5.2 Transform

Archivo: `dags/etl/transform.py`

Cuatro transformaciones puras (sin dependencias de Airflow — fácilmente testables):

a) Limpieza

```
def clean_sales(df: pd.DataFrame) -> pd.DataFrame:
    df = df.dropna(subset=["CustomerID"]) # elimina transacciones anónimas
    df = df[df["Quantity"] > 0]          # elimina cancelaciones (negativas y
    # cero)
    df = df[df["UnitPrice"] > 0]          # elimina ajustes y muestras gratuitas
    return df.copy()
```

Regla	Filas eliminadas (aprox.)	Motivo
CustomerID nulo	~135.080	No usables para análisis de clientes
Quantity ≤ 0	~10.624	Cancelaciones, no revenue

Regla	Filas eliminadas (aprox.)	Motivo
UnitPrice ≤ 0	~40	Ajustes contables

b) Normalización

```
def normalize_sales(df: pd.DataFrame) -> pd.DataFrame:
    df["Country"] = df["Country"].str.strip().str.lower()
    df["CustomerID"] = df["CustomerID"].str.strip().str.lower()
    df["country_key"] = df["Country"]
    return df
```

c) Enrichment con REST Countries

```
def enrich_with_countries(sales: pd.DataFrame, countries: pd.DataFrame) ->
pd.DataFrame:
    merged = sales.merge(countries, on="country_key", how="left")
    merged["region"] = merged["region"].fillna("unknown")
    merged["population"] = merged["population"].fillna(0).astype("int64")
    return merged
```

Se usa **left join** para no perder transacciones cuyo país no tiene equivalente en la API. Los países sin match (Channel Islands, EIRE, Unspecified) quedan con `region="unknown"` y se logea un warning con el número de filas afectadas.

d) Conversión de divisa

```
def apply_fx(df: pd.DataFrame, fx_rate: float) -> pd.DataFrame:
    df["revenue_gbp"] = (df["Quantity"] * df["UnitPrice"]).round(4)
    df["revenue_eur"] = (df["revenue_gbp"] * fx_rate).round(4)
    df["fx_rate_gbp_eur"] = fx_rate # guardado para auditoría
    return df
```

El tipo de cambio se almacena junto a cada registro para que la conversión sea auditable.

5.3 Load

Archivo: `dags/etl/load.py`

Creación de índices (primera ejecución)

```
coll.create_index(
    [("invoice_no", ASCENDING), ("stock_code", ASCENDING), ("customer_id",
```

```

ASCENDING)],
    unique=True, name="uq_business_key"
)
coll.create_index([("invoice_date", ASCENDING)], name="ix_invoice_date")
coll.create_index([("region", ASCENDING)], name="ix_region")

```

Carga en batches con upserts

```

for start in range(0, len(df), cfg.batch_size): # batch_size = 1000
    batch = df.iloc[start : start + cfg.batch_size]
    ops = [
        UpdateOne(
            {"invoice_no": rec["invoice_no"],
             "stock_code": rec["stock_code"],
             "customer_id": rec["customer_id"]},
            {"$set": rec},
            upsert=True
        )
        for rec in batch.to_dict(orient="records")
    ]
    result = coll.bulk_write(ops, ordered=False)

```

Actualización del watermark

```

meta.update_one(
    {"_id": "last_watermark"},
    {"$set": {"invoice_date": max_invoice_date, "updated_at": datetime.now(UTC)}},
    upsert=True,
)

```

El watermark se actualiza **solo tras el éxito de la carga completa**. Si la carga falla a mitad, el siguiente run reprocesa la misma ventana sin perder datos.

6. Extracción incremental — Watermarking

El pipeline implementa extracción incremental mediante un documento de metadatos en MongoDB:

Run	Watermark leído	Filas procesadas	Watermark escrito
1 (cold start)	∅ (ninguno)	~541.909 (todo el CSV)	<code>max(InvoiceDate)</code> = 2011-12-09
2 (datos nuevos)	2011-12-09	Solo filas posteriores	Nuevo máximo

Run	Watermark leído	Filas procesadas	Watermark escrito
3 (sin datos nuevos)	prev max	0	Sin cambios
4 (tras fallo parcial)	prev max	Misma ventana reprocesada	Actualizado tras éxito

Garantía: Gracias a la combinación de watermark + upserts idempotentes, el pipeline puede interrumpirse en cualquier punto y retomarse sin pérdida ni duplicación de datos.

7. Idempotencia — bulk_write vs insert_many

El problema

El enunciado del capstone pide dos requisitos simultáneamente:

1. Usar `insert_many()` en batches de 1000
2. Garantizar que re-ejecutar el pipeline no produce duplicados

Estos dos requisitos son **fundamentalmente incompatibles** con `insert_many`:

Solución con <code>insert_many</code>	Problema
<code>ordered=False</code> + ignorar errores de duplicado	Pérdida silenciosa de datos: en un retry parcial, los registros ya insertados se omiten sin posibilidad de distinguir entre "ya existía" y "rechazado por error real"
<code>delete_many</code> antes de <code>insert_many</code>	No atómico: existe una ventana donde la colección está vacía para ese rango de fechas. Un read concurrente (Power BI) devolvería datos incompletos

Nuestra solución: `bulk_write` + `UpdateOne(upsert=True)`

`bulk_write` es también una operación en batch — el tamaño del batch (1000) y el comportamiento de red son **idénticos** a `insert_many`. La diferencia es semántica:

- Si el documento **no existe** → se inserta (comportamiento idéntico a `insert`)
- Si el documento **ya existe** → se actualiza en el lugar (en vez de fallar o duplicar)

La clave de negocio (`invoice_no`, `stock_code`, `customer_id`) identifica unívocamente cada línea de factura. Un índice único compuesto sobre estos tres campos actúa como segunda línea de defensa a nivel de almacenamiento.

Contexto de industria

`bulk_write` con upserts es el patrón estándar para cargas ETL idempotentes en plataformas de datos con MongoDB. La documentación oficial de MongoDB lo recomienda explícitamente sobre `insert_many` para

pipelines que requieren idempotencia. Priorizamos **correctness** sobre la adherencia literal a la sugerencia de implementación del enunciado, documentando y justificando la decisión.

8. Monitorización y manejo de errores

Logging estructurado JSON

Todos los módulos usan un `JsonFormatter` personalizado que emite una línea JSON por evento:

```
{
  "ts": "2026-04-27T20:13:49+0000",
  "level": "INFO",
  "logger": "etl.extract",
  "msg": "task_start",
  "task": "extract"
}
{
  "ts": "2026-04-27T20:13:57+0000",
  "level": "INFO",
  "logger": "etl.extract",
  "msg": "csv_read",
  "rows": 541909
}
{
  "ts": "2026-04-27T20:13:58+0000",
  "level": "INFO",
  "logger": "etl.extract",
  "msg": "api_response",
  "api": "restcountries",
  "rows": 250
}
{
  "ts": "2026-04-27T20:13:58+0000",
  "level": "INFO",
  "logger": "etl.extract",
  "msg": "api_response",
  "api": "frankfurter",
  "rate": 1.1735
}
{
  "ts": "2026-04-27T20:14:01+0000",
  "level": "INFO",
  "logger": "etl.extract",
  "msg": "task_done",
  "task": "extract",
  "elapsed_sec": 12.4,
  "records": 541909
}
```

Decorador @timed

Cada función principal está decorada con `@timed(task_name)` que logea automáticamente:

- **Inicio de la tarea** con timestamp
- **Número de registros** procesados
- **Tiempo total** en segundos
- **Excepción** con traceback completo si falla

Estrategia de retries en dos capas

Capa	Mecanismo	Reintentos	Backoff
Llamadas a API (interna)	<code>tenacity</code>	5 intentos	Exponencial: 2s → 4s → 8s → 16s
Tarea de Airflow (externa)	<code>default_args retries</code>	2 reintentos	Exponencial hasta 15 min

Las dos capas protegen contra fallos diferentes: la interna absorbe flakiness HTTP transitorio; la externa cubre reinicios del contenedor o fallos del scheduler.

Manejo de errores en la carga

```
try:
    result = coll.bulk_write(ops, ordered=False)
except BulkWriteError as bwe:
    log.error("bulk_write_error", extra={
        "batch_start": start,
        "write_errors": bwe.details.get("writeErrors", [])[:3]
    })
```

```

}))
raise # propaga para que Airflow gestione el retry

```

9. Calidad de datos

La calidad se garantiza en tres capas ordenadas:

Capa 1 — Contrato de entrada (schema)

pandas `dtype` map falla rápido ante tipos inesperados. `parse_dates=["InvoiceDate"]` convierte la columna a timestamp real desde la lectura.

Capa 2 — Reglas de negocio (transform)

Regla	Implementación	Justificación
Eliminar CustomerID nulo	<code>dropna(subset=["CustomerID"])</code>	Transacciones anónimas no son usables para análisis de clientes
Eliminar Quantity ≤ 0	<code>df[df["Quantity"] > 0]</code>	Cancelaciones (negativas) y líneas vacías (cero) no representan revenue
Eliminar UnitPrice ≤ 0	<code>df[df["UnitPrice"] > 0]</code>	Ajustes contables y muestras gratuitas distorsionan el revenue
Normalizar Country/CustomerID	<code>str.strip().str.lower()</code>	Estabiliza joins y group-bys ante variaciones de capitalización/espacios

Capa 3 — Restricción de almacenamiento (load)

El índice único compuesto (`invoice_no`, `stock_code`, `customer_id`) en MongoDB rechaza duplicados a nivel de base de datos, independientemente de la lógica de aplicación.

Tests unitarios

Se implementaron **7 tests pytest** en `tests/test_transform.py` que cubren todas las funciones de transformación:

```

tests/test_transform.py::test_clean_removes_null_customer PASSED
tests/test_transform.py::test_clean_removes_negative_qty PASSED
tests/test_transform.py::test_clean_removes_zero_price PASSED
tests/test_transform.py::test_normalize_lowercases PASSED
tests/test_transform.py::test_enrich_adds_region_population PASSED
tests/test_transform.py::test_enrich_unknown_fallback PASSED
tests/test_transform.py::test_fx_calculation PASSED

===== 7 passed in 1.44s =====

```

10. Dashboard Power BI

Conexión a MongoDB Atlas

Power BI Desktop se conecta a Atlas mediante un script Python:

```
import pandas as pd
from pymongo import MongoClient

MONGODB_URI = "mongodb+srv://USER:PASSWORD@cluster0.xxxxx.mongodb.net/..."

client = MongoClient(MONGODB_URI, serverSelectionTimeoutMS=10_000)
coll = client["globalretail"]["fact_sales"]

projection = {"_id": 0, "invoice_no": 1, "stock_code": 1, "customer_id": 1,
              "invoice_date": 1, "quantity": 1, "unit_price_gbp": 1,
              "revenue_gbp": 1, "revenue_eur": 1, "country": 1,
              "region": 1, "population": 1}

fact_sales = pd.DataFrame(list(coll.find({}, projection)))
fact_sales["invoice_date"] =
pd.to_datetime(fact_sales["invoice_date"]).dt.tz_localize(None)
```

Nota: MongoDB Atlas M0 (Free Tier) no soporta el BI Connector SQL. El script Python es la alternativa recomendada para conectar Power BI directamente.

Configuración regional: Power BI debe estar configurado en **Inglés (Estados Unidos)** para interpretar correctamente el separador decimal punto (.) de los valores numéricos exportados por Python.

Visual 1 — Total Revenue by Region (Gráfico de barras)

Muestra el revenue total en EUR agregado por región geográfica.

Campo	Ubicación
region	Eje X (categoría)
Sum of revenue_eur	Eje Y (valor)

Insight: Europa representa ~95% del revenue total (~9.5M EUR), lo que refleja la base de clientes predominantemente británica del dataset.

Visual 2 — Revenue Trends over Months (Gráfico de líneas)

Muestra la evolución mensual del revenue a lo largo del año 2011.

Campo	Ubicación
invoice_date (Month de Date Hierarchy)	Eje X
Sum of revenue_eur	Eje Y

Insight: Crecimiento sostenido de agosto a noviembre con pico máximo en noviembre (~1.3M EUR), consistente con el comportamiento estacional del retail (Black Friday, pre-Navidad).

Visual 3 — High-Value Customer Analysis (Scatter Plot)

Identifica los clientes de mayor valor cruzando número de pedidos con revenue generado.

Medidas DAX creadas:

```
Orders = DISTINCTCOUNT(fact_sales[invoice_no])
Revenue EUR = SUM(fact_sales[revenue_eur])
```

Campo	Ubicación
customer_id	Valores (detalle por punto)
Orders	Eje X
Revenue EUR	Eje Y

Insight: El scatter revela dos segmentos claros — clientes de alto revenue con pocos pedidos grandes (esquina superior izquierda) y clientes frecuentes con ticket medio bajo (cluster inferior izquierdo).

11. Decisiones de diseño y trade-offs

Decisión	Trade-off
bulk_write+upsert en vez de insert_many	El enunciado sugiere insert_many pero es incompatible con idempotencia real. bulk_write+upsert satisface ambos requisitos con idéntico tamaño de batch. Ver §7 para análisis detallado.
Parquet en volumen compartido (no XCom)	Evita el anti-patrón de serializar 500k filas en la BD de metadatos de Airflow, que provocaría degradación del scheduler. El trade-off es dependencia de un volumen montado.
LocalExecutor (sin Celery/Redis)	Más simple y rápido de arrancar; suficiente para esta carga. En producción se usaría CeleryExecutor o KubernetesExecutor para escalar horizontalmente.
MongoDB Atlas M0 (Free Tier)	Gratuito y suficiente para el dataset (~387k documentos). Limitación: 512 MB de almacenamiento y sin BI Connector SQL. En producción se usaría M10+ o un warehouse columnar (Snowflake, BigQuery).
pandas en memoria	540k filas caben cómodamente en RAM (~100 MB). Para datasets >5M filas se usaría Polars o DuckDB sin cambiar la estructura del DAG.
FX rate por run (no por fecha de factura)	Cumple el requisito del enunciado ("current exchange rates"). La conversión históricamente precisa requeriría llamar al endpoint /v1/{date} de Frankfurter por cada fecha única del dataset.

Decisión	Trade-off
Left join para enrichment de países	Preferimos conservar transacciones con <code>region="unknown"</code> antes que descartarlas por un fallo de referencia. El conteo de misses se logea como warning para visibilidad.

12. Guía de reproducibilidad

Setup completo desde cero

```
# 1. Clonar/descargar el proyecto
cd globalretail-etl

# 2. Configurar variables de entorno
cp .env.example .env
# Editar .env: introducir MONGODB_URI de MongoDB Atlas

# 3. Colocar el dataset
# Descargar OnlineRetail.csv de Kaggle y copiarlo a:
# include/data/online_retail.csv

# 4. Construir imagen Docker
docker compose build

# 5. Inicializar Airflow (solo primera vez)
docker compose up airflow-init

# 6. Levantar servicios
docker compose up -d

# 7. Verificar estado
docker compose ps
# → postgres: healthy
# → airflow-webserver: healthy
# → airflow-scheduler: healthy

# 8. Abrir UI de Airflow
# http://localhost:8080 (admin / admin)
# Activar DAG → botón ► para ejecutar manualmente
```

Ejecutar tests unitarios

```
docker compose run --rm airflow-scheduler \
  bash -lc "cd /opt/airflow && pytest tests/ -v"
```

Verificar datos en MongoDB Atlas

Tras la primera ejecución, la colección `globalretail.fact_sales` debe contener ~387.841 documentos (540k filas originales menos las eliminadas en limpieza).

Conectar Power BI

1. Instalar dependencias: `pip install pymongo pandas dnspython`
 2. Power BI Desktop → **Obtener datos** → **Script de Python**
 3. Pegar el script de conexión con la `MONGODB_URI`
 4. Configurar región a **Inglés (Estados Unidos)** en Opciones
-

13. Conclusión

El pipeline ETL desarrollado satisface todos los requisitos del capstone brief y está construido con los mismos patrones que se usan en plataformas de datos reales:

- **Extracción incremental** mediante watermarking persistente en MongoDB
- **Idempotencia real** mediante upserts y un índice único compuesto
- **Resiliencia** mediante retries en dos capas (tenacity + Airflow)
- **Observabilidad** mediante logging estructurado JSON con métricas de tiempo y registros
- **Calidad de datos** garantizada en tres capas: schema, business rules y storage constraints
- **Orquestación moderna** mediante Apache Airflow con TaskFlow API en Docker

El dataset resultante en MongoDB Atlas (387.841 documentos, enriquecidos con región, población y revenue en EUR) alimenta un dashboard de Power BI con los tres visuales requeridos: Revenue por Región, Tendencia Mensual y Análisis de Clientes de Alto Valor.

Documento generado como parte del proyecto ETL Capstone — DAN2 2C — Loyola Universidad